

WORKING WITH PROCESSES IN LINUX

I. Objectives

- Students understand the concept of program, process, life cycle of a process, interprocess communication mechanism and basic system calls when working with processes.
- Students can using system calls and library functions for required tasks on processes handling.

II. Basic theory

1. Introduction to process

A **program** is a **set of instructions** written in a programming language that **performs a specific task** and passively stored on disk. A **process** is an **instance of a program** that is **being executed**. It is an active entity that runs in memory and utilizes system resources. While program and process are related, they are distinct concepts and be distinguished as below table:

Feature	Program	Process
Definition	A set of instructions stored on a disk	A program in execution, running in memory
State	Passive (not running)	Active (running)
Location	Stored on disk as an executable file	Loaded into RAM during execution
Lifespan	Permanent (until deleted)	Temporary (exists while executing)
Resource Usage	Does not use CPU, memory, or I/O	Uses CPU, memory, and I/O resources
Multiplicity	One copy of a program can exist on disk	Multiple instances (processes) of a program can run simultaneously

For an example, when you write and save a file named **hello.c**, it is just a text-based source code file (not a program or process). After you compile it using a compiler (e.g., `gcc hello.c -o hello`), it turns into an executable file (hello). This compiled executable file is the program—a passive entity stored on disk, waiting to be executed. When you run the compiled program by executing `./hello`, the operating system loads it into memory. At this point, the

OS creates a process, assigning system resources like CPU time, memory, and I/O handling. While it runs, the process exists in RAM and executes instructions from the program.

When a process is created, the Linux kernel allocates memory to it and organizes it in a structured way. The memory is divided into different segments to efficiently manage code execution, variables, stack operations, and dynamic memory. This table below summarizes memory layout of a process:

Segment	Description	Memory behavior	Example
Text (Code) Segment	Stores the compiled program's machine code (instructions).	Read-only to prevent modification.	<pre>int main() { return 0; // This machine code is stored in the text segment. }</pre>
Data Segment	Stores initialized global and static variables.	Fixed size, does not grow.	<pre>int x = 10; // Stored in the Data segment static int y = 20; // Also in Data segment</pre>
BSS (Block Started by Symbol)	Stores uninitialized global and static variables (automatically initialized to 0).	Fixed size, does not grow.	<pre>int uninitialized_var; // Stored in BSS segment, initialized to 0</pre>
Heap Segment	Stores dynamically allocated memory (e.g., malloc, new in C/C++).	Grows upward in memory.	<pre>int *ptr = (int*) malloc(sizeof(int)); // Allocated in Heap</pre>
Stack Segment	Stores function call frames, local variables, and return addresses.	Grows downward in memory.	<pre>void function() { int localVar; // Stored in Stack }</pre>

The kernel does not assign physical memory (RAM) directly to a process. Instead, it assigns a virtual address space, which acts as an abstraction of physical memory. Each process has its own virtual memory (isolated from other

processes). The Memory Management Unit (MMU), along with the **page table**, maps virtual memory addresses to actual physical memory (RAM) or swap space (disk).

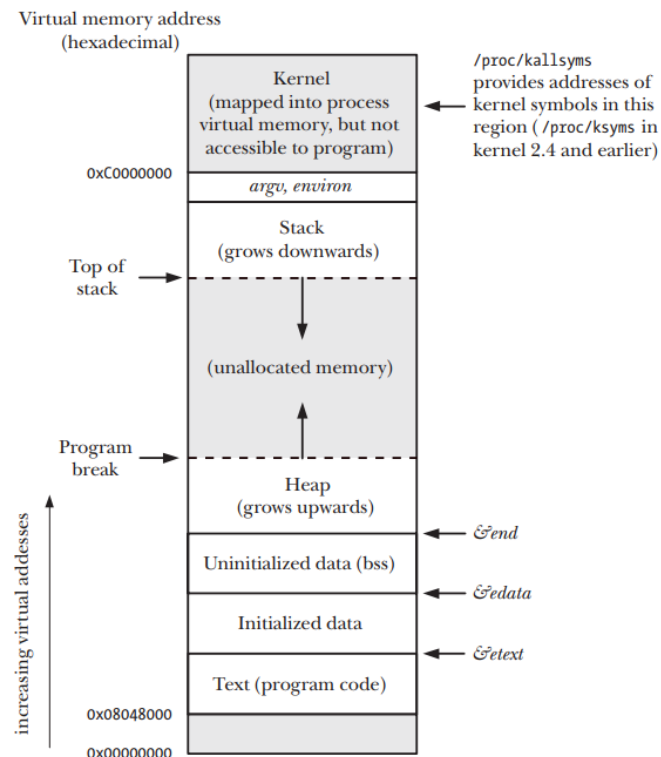
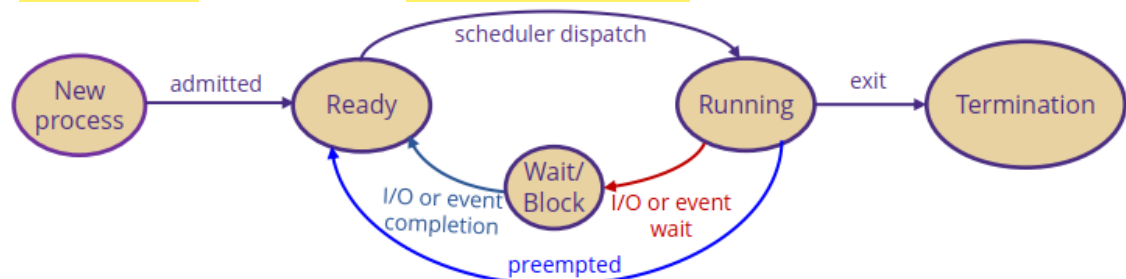


Figure 6-1: Typical memory layout of a process on Linux/x86-32

A process in Linux goes through different **states** in its life cycle: new, ready running, blocked (waiting) and terminated:

- New: Process is **created but not yet running**.
- Ready: Process is **waiting** in the ready **queue** for CPU time.
- Running: Process is **currently being executed** on the CPU.
- **Blocked** (Waiting): Process is **waiting for an event** (e.g., I/O operation).
- **Terminated**: Process has **finished execution**



2. Basic **system calls** when working with processes

In this section, we discover some **basic system calls** like **fork()**, **execve()**, **exit()**, **wait()**, which play a crucial role in creation, execution, and termination of a process in its life cycle. Before diving into that subject, we present a short overview of **fork()**, **execve()**, **exit()** and **wait()**.

2.1 Overview of **fork()**, **exit()**, **wait()**, and **execve()**

Each of system calls: **fork()**, **exit()**, **wait()**, and **execve()** has **variants**, which we'll also look at. For now, we provide an overview of these four system calls and how they are typically used together.

- The **fork()** system call **allows one process, the parent**, to **create a new process** the child. This is done by making the new child process an (almost) exact **duplicate of the parent**: the child obtains **copies of the parent's stack, data, heap and text segments**.
- The **exit(status)** library function **terminates a process**, making all resources (memory, open file descriptors, and so on) **used by the process available for subsequent reallocation by the kernel**. The **status** argument is an **integer** that determines the termination status for the process. **Using the wait() system call, the parent can retrieve this status**.
- The **wait(&status)** system call has two purposes. First, if a **child** of this process has **not yet terminated** by calling **exit()**, then **wait()** **suspends execution of the process until one of its children has terminated**. Second, the termination status of the child is **returned in the status** argument of **wait()**.
- The **execve(pathname, argv, envp)** system call **loads a new program (pathname, with argument list argv, and environment list envp) into a process's memory**. The existing program text is discarded, and the stack, data, and heap segments are freshly created for the new program. This operation is often **referred to as execing a new program**.

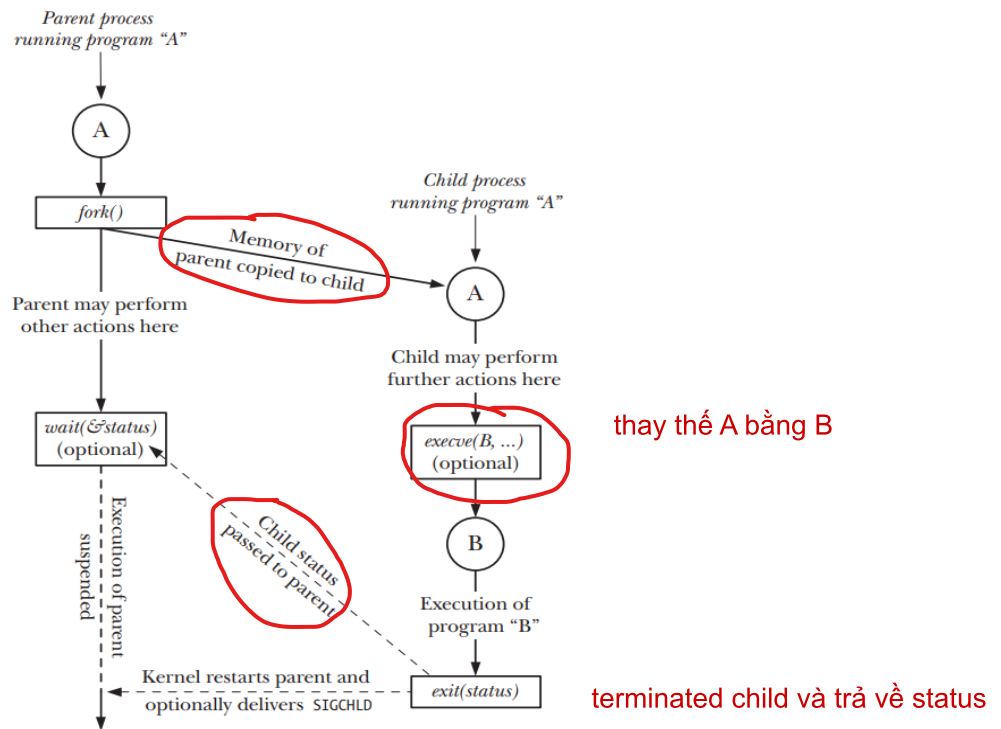


Figure 24-1: Overview of the use of `fork()`, `exit()`, `wait()`, and `execve()`

2.2. Creating a new process: `fork()`

The `fork()` system call creates a new process, the child, which is an almost exact duplicate of the calling process, the parent.

```
#include <unistd.h>
```

```
pid_t fork(void);
```

In parent: returns process ID of child on success, or -1 on error; in successfully created child: always returns 0

The key point to understanding `fork()` is to realize that after it has completed its work, two processes exist, and, in each process, execution continues from the point where `fork()` returns.

The two processes are executing the same program text, but they have separate copies of the stack, data, and heap segments. The child's stack, data, and heap segments are initially exact duplicates of the corresponding parts the parent's memory. After the `fork()`, each process can modify the variables in its stack, data, and heap segments without affecting the other process.

Within the code of a program, we can distinguish the two processes via the value returned from `fork()`. For the parent, `fork()` returns the process ID of the newly created child. For the child, `fork()` returns 0. If necessary, the child can obtain its own process ID using `getpid()`, and the process ID of

its **parent** using **getppid()**. If a new process can't be created, **fork()** returns -1.

It is also important to realize that after a **fork()**, it is indeterminate which of the two processes is next scheduled to use the CPU.

Example:

```
#include <stdio.h>
#include <unistd.h>

void main()
{
    pid_t childPid;    /* Used in parent after successful fork() to record PID of
    child */ switch (childPid = fork()) {
    case -1:            /* fork() failed */
        printf("Error when creating child process")    /* Handle error */
    case 0:            /* Child of successful fork() comes here */
        printf("I am child process");    /* Perform actions specific to child */
    default:            /* Parent comes here after successful fork() */
        printf("I am parent process");    /* Perform actions specific to parent */
    }
}
```

2.3. **Waiting** on the child process

★ THE **wait()** SYSTEM CALL

The **wait()** system call **waits** for one of the **children** of the calling process to terminate and **returns the termination status** of that child in the buffer pointed to by **status**.

```
#include <sys/wait.h>
```

```
pid_t wait(int *status);
```

Return value:

- Returns the **process ID (PID)** of the **terminated child**.
- Returns **-1** if an error occurs.

Functionality of **wait()**

- **If no child process has yet terminated, wait() blocks the parent process until a child terminates.**

- If a child process has **already terminated before wait()** is called, it **returns immediately**.
- If **status is not NULL**, information about the child's termination is **stored in it**.
- The kernel accumulates the child's CPU time and resource usage statistics into the parent process.
- If the parent process has **no child to wait for**, **wait()** **returns -1** and sets **errno = ECHILD**.

Key Considerations

- **Zombie Processes**: If a parent does not call wait(), terminated children become zombie processes, occupying system resources.
- **Multiple Child Processes**: If several children terminate at the same time, the order in which they are reaped is **unspecified** and may **vary across different implementations**.
- **Error Handling**: Always check the return value of wait(). If **-1** is returned with **errno == ECHILD**, it means there are **no more children**.

★ THE **WAITPID()** SYSTEM CALL

The **waitpid()** system call is an **improved version of wait()**, designed to provide more control when waiting for child processes. It allows **selecting a specific child to wait for**, performing non-blocking waits, and monitoring child process state changes.

```
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *status, int options);
```

Return value:

- **Process ID** of the waited-for child
- **0** in non-blocking mode
- **-1** on error.

Parameters:

- **pid**: Specifies which child process to wait for.
- **status**: Pointer to an integer storing child termination details.
- **options**: Bitmask flags for additional behavior control.

Arguments value:

PID

pid > 0	Wait for a child with the specified process ID
pid == 0	Wait for any child in the same process group
pid > -1	Wait for any child in the process group specified by -pid
pid == -1	Wait for any child (equivalent to wait())

OPTIONS

WUNTRACED	Return status when a child is stopped by a signal
WCONTINUED	Return status when a stopped child is resumed (SIGCONT)
WNOHANG	Perform a non-blocking check (returns 0 if no child has changed state)

Key Considerations:

- Unlike **wait()**, which **waits** for **any child** to terminate, **waitpid()** can **wait** for a **specific child process**.
- Using the **WNOHANG** option, **waitpid()** can check for child process status **without blocking execution**.
- **waitpid()** can **return information** when a child is stopped (**WUNTRACED**) or resumed (**WCONTINUED**), not just terminated.

★ THE **WAIT STATUS** VALUE

When a child process changes state, **wait()** and **waitpid()** return a wait status value. This value helps determine:

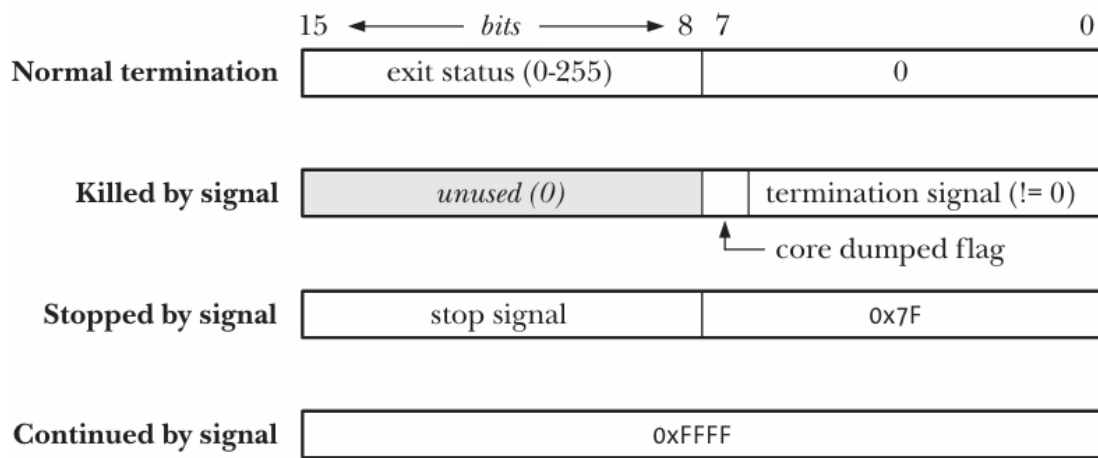
- If the child terminated normally (via **_exit()** or **exit()**).

- If the child was killed by an unhandled signal.
- If the child was stopped by a signal (WUNTRACED flag in waitpid()).
- If the child resumed after receiving SIGCONT (WCONTINUED flag in waitpid()).

This wait status is stored in the lower 2 bytes of an int value and should be examined using standard macros instead of direct bit manipulation.

The wait status value consists of:

- Exit status (0-255) if the child terminated normally.
- Termination signal if the child was killed by a signal.
- Core dump flag (if applicable).
- Stop signal (if the child was stopped).



Since this structure may vary across systems, applications should always use macros to interpret the value.

MACROS FOR INSPECTING WAIT STATUS

The <sys/wait.h> header provides macros to analyze the wait status value:

Normal Termination	WIFEXITED(status)	- Returns true if the child exited normally. - Use WEXITSTATUS(status) to get the child's exit code (0-255).
Killed by a	WIFSIGNALED(status)	- Returns true if the child was

Signal		terminated by a signal. - Use WTERMSIG(status) to get the signal number. - Use WCOREDUMP(status) to check if a core dump was produced (not part of SUSv3 but widely available).
Stopped by a Signal	WIFSTOPPED(status)	- Returns true if the child was stopped by a signal (e.g., SIGSTOP, SIGTSTP). - Use WSTOPSIG(status) to get the signal number.
Resumed after SIGCONT	WIFCONTINUED(status)	- Returns true if the child was resumed with SIGCONT. - Available since Linux 2.6.10.

Example:

```

#include <stdio.h> /* for printf() */
#include <unistd.h> /* for fork() */
#include <sys/wait.h> /* for waitpid() */

int main() {
    int status;
    pid_t child_pid = fork();

    if (child_pid > 0) {
        //Parent process
        waitpid(child_pid, &status, 0);
        printf("I am the parent\n");
    } else if (child_pid == 0) {
        // Child process
        printf("I am the child\n");
    }
}

```

```

    } else {
        // fork() failed
        printf("Fork failed!\n");
    }

    return 0;
}

```

In the above code, it may happen that the parent process prints "I am the parent" before the child process prints "I am the child". To ensure that the child process always prints first, you can add `sleep(1);` in parent process:

```

if (child_pid > 0) {
    sleep(1); //Wait child process print first
    waitpid(child_pid, &status, 0);
    printf("I am the parent\n");
}

```

2.5 PROGRAM EXECUTION

2.5.1. Executing new program: `execve()`

The `execve()` system call loads a new program into a process's memory, replacing its existing program. The old program's stack, data, and heap are discarded. The new program starts execution from its `main()` function, after any necessary runtime initializations.

```

#include <unistd.h>

int execve(const char *pathname, char *const argv[], char *const envp[]);

```

Returns:

- Never returns on success (the process is replaced).
- Returns -1 on error, with `errno` indicating the failure reason.

Parameters of `execve()`:

pathname (Program to Execute)	- Specifies the absolute or relative path of the new program to execute. - Can be an ELF executable or a script with a shebang (#!).
argv[] (Command-Line Arguments)	- Specifies the arguments passed to the new program (similar to argv in main()). - argv[0] typically contains the program name (basename of pathname). - Must be NULL-terminated.
envp[] (Environment Variables)	- Defines the environment for the new program. - Contains key-value pairs like "PATH=/usr/bin". - Must be NULL-terminated.

- The **process ID (PID) remains the same** since the process itself is not replaced, only its memory contents.
- **Some** process **attributes remain** unchanged (e.g., process ID, file descriptors, signal handlers), while **others are reset**.
- If the program file has the set-user-ID (SUID) or set-group-ID (SGID) bit set, the process's effective user/group ID is changed to match the file owner.
- If execve() is successful, the old program is completely replaced and **does not return**.
- If an error occurs, execve() **returns -1**, allowing the caller to check errno for the failure reason.

★ ERROR HANDLING IN `execve()`

các mã lỗi khi dùng `execve()`

Common reasons `execve()` may fail:

EACCES	File is not executable, lacks execution permission, or resides on a MS_NOEXEC filesystem.
ENOENT	File does not exist.
ENOEXEC	File is marked executable but is in an unrecognized format (e.g., a script without #!).

ETXTBSY	File is currently open for writing by another process.
E2BIG	The total space required for arguments and environment exceeds system limits.

These errors can also apply to interpreter files used for executing scripts or ELF binaries.

★ `execve()` AND ELF EXECUTABLES

- The **Executable** and **Linking Format (ELF)** defines **how executable files** are structured.
- Normally, `execve()` **loads segments** from the **executable file** into memory.
- Some **ELF files** specify an **interpreter** (`PT_INTERP`), meaning the kernel loads the interpreter first, which then executes the program.
- **This allows for dynamic linking and execution of interpreted languages.**

2.5.2 The `exec()` library function

The `exec()` **family of functions** provides different ways to **replace a process's memory** with a **new program**. These functions are built on `execve()` but offer various options for specifying the program **name, arguments, and environment**.

Each function differs in how it specifies the **program file, arguments, and environment**. The **function names** provide clues to these differences:

- `p (PATH)` → Searches for the program in the `PATH` environment variable.
- `l (list)` → Accepts arguments as a variable-length list.
- `v (vector)` → Accepts arguments as a `NULL`-terminated array.
- `e (environment)` → Allows specifying a custom environment.

```
#include <unistd.h>

int execve(const char *pathname, char *const argv[], char *const envp[]);
int execl(const char *pathname, const char *arg, ... /*, (char *) NULL, char
*const envp[] */);
```

```

int execlp(const char *filename, const char *arg, ... /* , (char *) NULL */);
int execvp(const char *filename, char *const argv[]);
int execv(const char *pathname, char *const argv[]);
int execl(const char *pathname, const char *arg, ... /* , (char *) NULL */);

```

- All functions never return on success (the new program takes over).
- They return -1 on error, with errno set accordingly.

★ DIFFERENCES BETWEEN `exec()` VARIANTS

Function	Program File Specified By	Arguments Specified As	Environment
<code>execve()</code>	Absolute or relative pathname	Array (<code>argv[]</code>)	Custom (<code>envp[]</code>)
<code>execle()</code>	Absolute or relative pathname	List (<code>arg1, arg2, ..., NULL</code>)	Custom (<code>envp[]</code>)
<code>execlp()</code>	Filename, searches PATH	List (<code>arg1, arg2, ..., NULL</code>)	Uses caller's environment
<code>execvp()</code>	Filename, searches PATH	Array (<code>argv[]</code>)	Uses caller's environment
<code>execv()</code>	Absolute or relative pathname	Array (<code>argv[]</code>)	Uses caller's environment
<code>execl()</code>	Absolute or relative pathname	List (<code>arg1, arg2, ..., NULL</code>)	Uses caller's environment

★ Function Variants

- Functions that use a pathname (`execve()`, `execle()`, `execv()`, `execl()`)
 - The program must be specified using an absolute or relative pathname (e.g., `/bin/lis` or `./my_program`).
 - Does not search PATH.
- Functions that search the PATH variable (`execlp()`, `execvp()`)

- The program can be specified by filename only (e.g., "ls").
- Searches for the program in directories listed in the PATH environment variable.
- If the filename contains /, PATH is not used (it is treated as a direct pathname).
- Functions that specify arguments as a list (execl(), execlp(), execl_e())
 - The argument list is passed directly in the function call (arg1, arg2, ..., NULL).
 - NULL must terminate the list.
- Functions that specify arguments as an array (execve(), execvp(), execv())
 - The argument list is passed as a NULL-terminated array (argv[]).
 - Easier to use for dynamic argument lists.
- Functions that allow a custom environment (execve(), execl_e())
 - envp[] specifies the new environment.
 - Other functions inherit the parent's environment.

Example:

```
#include <stdio.h>
#include <unistd.h>
void main() {
    static char *cmd[]={“ps”, “ls”, “date”, “goof”}; int i;
    printf(“0=ps, 1=ls, 2=date : ”); scanf(“%d”,&i);
    execlp(cmd[i], cmd[i], NULL);
    printf(“command not found\n”); }
```

2.6. Terminating a Process: _exit() and exit()

A process may terminate in two general ways. One of these is abnormal termination, caused by the delivery of a signal whose default action is to terminate the process. Alternatively, a process can terminate normally, using the _exit() system call.

```
#include <unistd.h>
```

chỉ dùng 1 byte cuối để lưu các mã trạng thái

```
void _exit(int status);
```

The **status argument** given to `_exit()` defines the termination status of the process, which is **available to the parent** of this process when it calls `wait()`. Although **defined as an int**, **only the bottom 8 bits** of status are actually made available to the parent. By convention, a **termination status of 0** indicates that a process **completed successfully**, and a **nonzero status value indicates that the process terminated unsuccessfully**. There are no fixed rules about how nonzero status values are to be interpreted. Programs generally don't call `_exit()` directly, but instead call the `exit()` library function, which performs various actions before calling `_exit()`.

```
#include <stdlib.h>
void exit(int status);
```

The following actions are performed by `exit()`:

- Exit handlers (functions registered with `atexit()` and `on_exit()`) are called, in reverse order of their registration.
- The stdio stream buffers are flushed.
- The `_exit()` system call is invoked, using the value supplied in status

3. Interprocess communication (IPC)

For **processes communicating with each other**, there are two key mechanism: **signal and pipe**.

Signal

Signals are small messages that **notify a process when an event occurs** in the system. They **provide a way** for processes to **communicate**, usually in an **asynchronous manner**, which means that a process might **receive a signal at any time**, regardless of what it was doing. Signals are commonly **generated by: kernel**, when an **event** like an error or **timer expiration** occurs; **other processes** (using system calls like `kill()`) and the **process itself**, for handling conditions such as division by zero.

When a signal is sent, the **kernel interrupts** the target process and takes one of two actions. **If the process has registered a signal handler**, that function is **executed**. If no handler is specified, the **default action for the signal is performed**. Table below covers some common signals and their default actions:

ID	Name	Default action	Event trigger
2	SIGINT	Terminate	Ctrl + C (keyboard interrupt)
9	SIGKILL	Terminate	Kills a process (cannot be ignored)

11	SIGSEGV	Terminate & Dump	Segmentation fault (invalid memory access)
14	SIGALRM	Terminate	Timer expires
17	SIGCHLD	Ignore	Child process stops or terminates

The `kill()` system call in Linux is used to **send a signal** from one process to **another**. Syntax for `kill()`:

```
#include <signal.h>
int kill(pid_t pid, int sig);
```

pid: The process ID (**PID**) of the target process.

- If `pid > 0`, the signal is sent to the specific process.
- If `pid == 0`, the signal is **sent to all processes** in the current **process group**.
- If `pid == -1`, the signal is **sent to all processes** **except the calling process** and `init`.
- If `pid < -1`, the signal is sent to all processes in the process group `|pid|`.

sig: The signal number (e.g., `SIGTERM`, `SIGKILL`, `SIGUSR1`)

Example:

```
kill(pid, SIGKILL)
```

Interrupt Service Routine

In Linux, processes can handle signals by associating a signal handler function using the `signal()` system call. This **allows a process to define custom behavior** when it **receives specific signals**, instead of using the default action. The `signal()` function is used to set up a signal handler for a specific signal.

```
#include <signal.h>
void (*signal(int signum, void (*handler)(int)))(int);
```

Parameters:

- `signum`: The signal number (e.g., `SIGINT`, `SIGTERM`, `SIGUSR1`).
- `handler`: A **function pointer** to the **signal handler function**.

Return value:

- On success, `signal()` **returns the previous handler function**.
- On **failure**, it returns `SIG_ERR`.

By default, most signals either: terminate the process (e.g., `SIGTERM`, `SIGSEGV`), ignore the signal (e.g., `SIGCHLD`) or stop/resume the process

(e.g., SIGSTOP, SIGCONT). Using signal(), a process can override these default behaviors and specify a custom function.

Note: Signal handlers execute asynchronously, they can interrupt a process at any time. Not all signals can be caught (SIGKILL (9) and SIGSTOP (19) cannot be handled or ignored.)

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Custom signal handler function
void handle_sigint(int sig) {
    printf("\nReceived SIGINT (Ctrl+C). Ignoring it!\n");
}

int main() {
    // Associate SIGINT with the handler function
    signal(SIGINT, handle_sigint);

    while (1) {
        printf("Running... Press Ctrl+C to test signal handling.\n");
        sleep(2);
    }
    return 0;
}
```

Pipe

A pipe is a **unidirectional interprocess communication mechanism** that allows **data to flow between a producer** (writing process) **and a consumer** (reading process). It functions as a **buffer between processes**. **Producer writes data into the pipe and Consumer reads data from the pipe**. If the pipe is full, the **producer blocks (waits)** until there is space. If the **pipe is empty**, the consumer blocks until data is available.

There are two types of pipes:

- **Anonymous Pipes:** Only work between related processes (like parent and child) and be created using **pipe() system call**.
- **Named Pipes (FIFOs):** Allow communication between any two processes and can be created using **mkfifo()**.

Pipe() system call is used to **create an anonymous pipe** which allows data to flow unidirectionally between two related processes with syntax:

```
#include <unistd.h>
int pipe(int fd[2]);
```

Parameters:

fd[2] → An integer array where:

- fd[0]: Read end of the pipe (used by the consumer).
- fd[1]: Write end of the pipe (used by the producer).

Return value:

- Returns 0 on success.
- Returns -1 on failure (and sets errno).

Example:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2]; // File descriptors for pipe
    char buffer[50];

    if (pipe(fd) == -1) {
        perror("Pipe failed");
        return 1;
    }

    if (fork() == 0) { // Child process (Consumer)
        close(fd[1]); // Close unused write end
        read(fd[0], buffer, sizeof(buffer)); // Read from pipe
        printf("Child received: %s\n", buffer);
        close(fd[0]); // Close read end
    } else { // Parent process (Producer)
        close(fd[0]); // Close unused read end
        char message[] = "Hello from parent!";
        write(fd[1], message, strlen(message) + 1); // Write to pipe
        close(fd[1]); // Close write end
    }
    return 0;
}
```

In Linux, `dup()` and `dup2()` are system calls used to **duplicate file descriptors**. They are commonly used in interprocess communication (IPC) to redirect input/output between processes, especially when using pipes.

dup() system call

```
#include <unistd.h>
int dup(int oldfd);
```

`dup()` system call **creates a copy of oldfd (an existing file descriptor)** and **returns a new file descriptor** that refers to the same file. The new descriptor uses the lowest available file descriptor number. Both file descriptors share the same file offset.

dup2() system call

```
#include <unistd.h>
int dup2(int oldfd, int newfd);
```

`dup2()` duplicates `oldfd` and assigns it to `newfd`. If `newfd` is already open, it is closed first before duplication. It is useful when redirecting standard input (`stdin`), output (`stdout`), or error (`stderr`).

Example: This program redirects `stdout` (`fd = 1`) to a file using `dup2()`

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("File open failed");
        return 1;
    }

    dup2(fd, STDOUT_FILENO); // Redirect stdout to file
    close(fd);

    printf("This message will be written to output.txt instead of the
terminal.\n");
    return 0;
}
```